

The Memory and Configuration Hierarchy

Where CLAUDE.md files live, why they do not override each other, and where the strict precedence chain actually is.

Settings are **what Claude can do**. Memory files are **what Claude should know**.

Where memory files live

FILE	LOCATION	WHO GETS IT
Managed policy	A system-wide managed location	Everyone in the organisation
User memory	~/.claude/CLAUDE.md	You, in every project
Project memory	./CLAUDE.md at the repo root	Anyone who clones the repo
Directory memory	CLAUDE.md in a subdirectory	Loaded when work happens there
Personal project memory	./CLAUDE.local.md, gitignored	Just you, just this project

ON THE EXAM

Claude reads CLAUDE.md files up the directory tree. A monorepo can have one per component.

The fact most guides get wrong

Memory files are concatenated. They are not a precedence chain.

- ◆ **All the loaded CLAUDE.md files are joined into context.** There is no documented override rule between them.
- ◆ **"More specific wins" and "project overrides user" are paraphrases the docs do not make.**
- ◆ **CLAUDE.local.md is appended after CLAUDE.md at the same level.** That is load order, not precedence. Reading last does not win a contradiction.
- ◆ **So if two memory files contradict each other, the outcome is not guaranteed.** Fix the contradiction; do not rely on a winner.

ON THE EXAM

If a question asks which CLAUDE.md wins, the honest answer is neither reliably. Move the rule to settings.json or a hook.

Two hierarchies, two behaviours

CLAUDE.MD

Concatenated context

- Files are joined, not ranked
- No documented conflict resolution
- Contradictions resolve unpredictably
- Fix by not contradicting yourself

SETTINGS.JSON

A strict precedence chain

- Five layers, merged low to high
- Managed policy always wins
- Within a layer: deny beats ask beats allow
- Predictable, testable, auditable

THE TIEBREAKER

This is why enforcement lives in settings. It is the only one of the two with a defined answer to "which wins?".

What belongs in a memory file

- ◆ **Conventions and standards.** Naming, structure, the testing approach, the commit format.
- ◆ **Architecture and why.** The reason that directory is odd is worth more than the fact that it is.
- ◆ **Commands that are not obvious.** How to run the tests, how to start the dev server.
- ◆ **Things a new engineer would ask on day one.** That is a good filter for the whole file.
- ◆ **Not secrets, and not hard rules.** Both belong somewhere else.

ON THE EXAM

Write it for a competent new colleague who will not read your codebase first.

Imports, and what they cost

- ◆ **@path pulls another file in.** Relative paths resolve from the importing file, not your working directory.
- ◆ **Everything imported is loaded into context every session.** An import is not a lazy reference.
- ◆ Imports outside your working directory trigger an **approval dialog** the first time.
- ◆ **Not evaluated inside code blocks or code spans**, so an @ in an example is safe.

ON THE EXAM

Splitting one big file into imported files does not reduce what loads. It only reorganises it.

Path-scoped rules

- ◆ Files under `.claude/rules/` can declare which paths they apply to, in frontmatter.
- ◆ They load when work touches those paths, rather than on every session.
- ◆ This is what actually reduces what loads. Imports do not; path scoping does.
- ◆ Good for monorepos, where the frontend conventions are noise during backend work.

ON THE EXAM

One big CLAUDE.md that is 80% irrelevant to today's task is a path-scoping problem.

CLAUDE.local.md and the worktree trap

- ◆ **For private, per-project preferences.** Sandbox URLs, your test data, local ports.
- ◆ **Gitignored, so it never reaches the team.** That is the point of it.
- ◆ **A gitignored file exists only in the worktree where you made it.** Multiple worktrees means multiple copies to maintain.
- ◆ **Import from your home directory instead,** so one file serves every worktree.

ON THE EXAM

"It works in one checkout and not the other" is almost always this.

The settings precedence chain

LAYER	BEATS
Enterprise managed policy	Everything. Nothing below can relax it.
CLI flags	Local, project and user settings
Project <code>.claude/settings.local.json</code>	Project and user settings
Project <code>.claude/settings.json</code>	User settings
User <code>~/.claude/settings.json</code>	The baseline

ON THE EXAM

Read it top down: managed is the policy floor. Read it bottom up: user settings are defaults the project can override.

Inside a layer: strictest wins

- ◆ Permissions have three modes: allow, deny and ask.
- ◆ Within any single layer, deny beats ask, and ask beats allow. The strictest matching rule wins.
- ◆ So an allow rule cannot rescue something denied in the same layer.
- ◆ To override a project deny on your own machine, put the allow in settings.local.json, which is a higher layer.

ON THE EXAM

Two rules combine here: layers override downwards, and inside a layer the strictest match wins. Questions test both.

Permissions in practice

- ◆ **A deny on reading .env files** keeps secrets out of the model's context, cleanly and provably.
- ◆ **Deny is a capability removal.** No prompt, no reasoning, no exception.
- ◆ **Ask puts a human in the loop** for things that are sometimes right.
- ◆ **Permissions are on or off.** Conditional rules need a hook, which module 9 covers.

ON THE EXAM

This is the same enforcement lesson from module 6, now with a file path attached to it.

When a settings change takes effect

- ◆ **Most settings reload automatically.** Permissions and hooks apply as soon as you save.
- ◆ **A few are read once at startup,** including the model and the output style.
- ◆ **"I changed it and nothing happened" is usually one of those,** and a restart fixes it.
- ◆ **claude doctor validates that your settings file is being read at all.**

ON THE EXAM

Distinguish "the setting is wrong" from "the setting has not been picked up yet". They look identical.

Working out what actually loaded

COMMAND	WHAT IT TELLS YOU
/context	Which memory files actually loaded into this session
/memory	Lists your memory file locations and opens them for editing
/status	Version, session ID, model, and which settings sources are active
claude doctor	Whether your configuration files are being read correctly

ON THE EXAM

Neither `/context` nor `/memory` loads anything. They report on loading that already happened automatically.

Currency check: the diagnostic commands split

TOPIC	EXAM ANSWER	CURRENT REALITY
"Which memory files are loaded?"	/memory	/context reports what loaded. /memory lists and opens files.
Auto memory	Does not exist	Claude Code writes its own per-project notes, outside the concatenation order.
AGENTS.md	Not mentioned	Interoperability with the cross-tool convention now exists.

ON THE EXAM

Answer /memory on the exam. Use /context at your desk. This was flagged in the introduction; here is the detail.

Reading the stem: configuration questions

WHAT THE STEM SAYS	WHAT IT WANTS
"must hold regardless of what Claude decides"	settings.json deny, or a hook. Never a memory file.
"the whole team should get this"	Project CLAUDE.md, or project settings.json
"just my machine, not committed"	CLAUDE.local.md, or settings.local.json
"the memory file has become unmanageable"	Path-scoped rules under .claude/rules/
"two files say different things"	Concatenated. No guaranteed winner. Move the rule.
"IT policy must not be overridable"	Enterprise managed settings

ON THE EXAM

Almost every row is decided by one question: does this need to be known, or enforced?

Module 8 in five lines

- ◆ Settings are what Claude can do. Memory files are what Claude should know.
- ◆ Memory files are concatenated, not ranked. Contradictions have no guaranteed winner.
- ◆ settings.json has five layers. Managed policy always wins; inside a layer, deny beats ask beats allow.
- ◆ Imports always load. Path-scoped rules under .claude/rules/ are what reduce context.
- ◆ /context tells you what loaded. Nothing you run loads anything; loading is automatic.

ON THE EXAM

Five scenarios next. One of them turns entirely on the concatenation fact.

A team's project CLAUDE.md says "always use four-space indentation". One engineer's user-level CLAUDE.md says "always use two-space indentation".

That engineer reports that Claude uses four spaces most of the time, two spaces occasionally, and once mixed both within a single file. They want the project standard to apply reliably.

What should they do?

- A Nothing. Project-level memory overrides user-level memory, so four spaces already wins
- B Remove the conflicting rule from the user CLAUDE.md, and enforce formatting with a hook or a formatter
- C Move the four-space rule into CLAUDE.local.md so it is read last
- D Add stronger wording to the project CLAUDE.md, such as "this overrides any personal preference"

◆ PAUSE AND DECIDE

What should they do?

A Project overrides user

The claim the docs never make. Memory files are concatenated, and the observed behaviour disproves it anyway.

C Read last wins

Load order is not precedence. It also puts a team standard in a personal gitignored file.

B Remove the conflict, enforce with a hook

Ends the contradiction, then puts formatting where it has a defined outcome.

D Stronger wording

Two contradictory instructions in one context window. Emphasis does not resolve that reliably.

THE SIGNAL

"Four spaces most of the time, two occasionally, once both" is the signature of concatenation with no override rule.

A monorepo has a single CLAUDE.md of about 900 lines covering the React frontend, the Python services, the Terraform infrastructure and the data pipeline.

Engineers report that a large amount of context is consumed before any work begins, and that Claude sometimes applies frontend conventions to Python files.

One engineer proposes splitting it into four files and importing them all from the root CLAUDE.md.

What is the most effective change?

- | | |
|---|---|
| <p>A Split into four files and import them from the root CLAUDE.md as proposed</p> | <p>B Split into path-scoped rules under <code>.claude/rules/</code> so each loads only when work touches its paths</p> |
| <p>C Move the frontend and infrastructure sections into CLAUDE.local.md</p> | <p>D Add a line to the CLAUDE.md instructing Claude to apply only the relevant section</p> |

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Split and import
Tidier to read, identical to load. All four imports resolve into context every session.
- B

 Path-scoped rules
Only the relevant conventions load, which fixes the context cost and the cross-contamination together.
- C

 Into CLAUDE.local.md
Same volume, now gitignored, so the team loses shared conventions. Worse on every axis.
- D

 Instruct it to be selective
The content is already in context. Asking it to ignore what it can see is not context management.

THE SIGNAL

Two symptoms, one cause. Both the token cost and the wrong-conventions problem come from everything being loaded always.

A team's project settings.json denies Bash(curl *). One engineer legitimately needs curl for local testing against a sandbox API.

They add an allow rule for Bash(curl *) to the same project settings.json file. It has no effect.

They cannot change the team's policy and do not want to commit a change that affects everyone.

What should they do?

- ☐ A Add the allow rule to .claude/settings.local.json instead
- ☐ B Add the allow rule to their user ~/.claude/settings.json
- ☐ C Remove the deny rule from the project settings.json
- ☐ D Add a note to CLAUDE.local.md explaining that curl is permitted for this engineer

◆ PAUSE AND DECIDE

What should they do?

- A

 settings.local.json
A higher layer than project settings, and uncommitted. It overrides the project deny for this machine only.
- B

 User settings
User is the lowest layer. Project settings override it, so the deny still wins.
- C

 Remove the project deny
Changes the policy for the whole team, which the stem explicitly rules out.
- D

 A note in CLAUDE.local.md
A memory file cannot grant a permission. Context does not override a deny rule.

THE SIGNAL

The allow failed because deny beats allow inside a layer. Getting past it means moving up a layer, not writing a louder rule.

An engineer adds a rule to their project CLAUDE.md and reports that Claude is ignoring it entirely. They have restarted the session twice.

A colleague asks how they know the file is being loaded. The engineer says they can see the file on disk with the rule in it.

What is the right next step?

- ☐ A Run `/context` and check the list of memory files that loaded this session
- ☐ B Run `/memory` to load the file into the session
- ☐ C Restate the rule more forcefully and restart again
- ☐ D Move the rule to the user-level CLAUDE.md, which loads more reliably

◆ PAUSE AND DECIDE

What is the right next step?

- A

 /context to verify loading
Separates "the rule is being ignored" from "the file never loaded". Those need opposite fixes.
- B

 /memory to load it
Nothing loads a memory file on demand. Loading is automatic; the commands only report.
- C

 Say it more forcefully
Skips diagnosis entirely. If the file never loaded, the wording is irrelevant.
- D

 Move it to user level
Invents a reliability difference between levels. Also makes a project rule personal.

THE SIGNAL

Existing on disk and being loaded are different claims. The engineer has verified the first and assumed the second.

A regulated organisation must guarantee that no Claude Code session anywhere in the company can read files matching *.pem or *.key, and must be able to demonstrate this to an auditor.

Engineering teams control their own repositories and their own settings files. Some engineers use `--dangerously-skip-permissions` locally.

What satisfies the requirement?

- ☐ A A deny rule in every project's `.claude/settings.json`, enforced by code review
- ☐ B A deny rule in the enterprise managed settings file, deployed by IT
- ☐ C A line in the organisation-wide `CLAUDE.md` forbidding access to key material
- ☐ D A deny rule in each engineer's user settings, distributed by a setup script

◆ PAUSE AND DECIDE

What satisfies the requirement?

- A

 Per-project deny plus review
Every repository is a place to forget. Review catches most, and "guarantee" is not most.
- B

 Enterprise managed settings
The only layer nothing below can relax. It is the policy floor, and it is demonstrable.
- C

 Organisation-wide CLAUDE.md
A memory file. Concatenated context with no guaranteed outcome, and nothing to show an auditor.
- D

 User settings by script
The lowest layer of the five. A project or local setting overrides it, and scripts drift.

THE SIGNAL

"Guarantee", "anywhere in the company" and "demonstrate to an auditor" all point at the same single layer.